

Resolución de la Actividad 4: Clase Virtual 02

Grupo 4

Integrantes:

- Avila, Aldana, legajo 1193721, alavila@uade.edu.ar
- Caivano, Alexander Francisco, legajo 1209389, acaivano@uade.edu.ar
- Casareski, Juan Ignacio, legajo 1176109, jcasareski@uade.edu.ar
- Segura, Juan Ignacio, legajo 1242159, jsegura@uade.edu.ar

Escenario base utilizado

Variable	Valor inicial	Valor proyectado (1 año)
Tenants activos	50	300
Procesos promedio por tenant	10	25
Instancias diarias por tenant	200	800
Eventos promedio por instancia	15	25
Usuarios concurrentes promedio	300	2.000
Pico de eventos por segundo	300	2.000

1. Parte A: Estimación de volumen

Cálculos

Escenario inicial - Instancias diarias totales = 50 tenants × 200 instancias = 10.000 instancias/día - Eventos diarios totales = 10.000 × 15 = 150.000 eventos/día - Eventos mensuales (×30) = 4.500.000 eventos/mes - Definiciones de proceso = 50 × 10 = 500 definiciones

Escenario proyectado - Instancias diarias totales = 300 × 800 = 240.000 instancias/día - Eventos diarios totales = 240.000 × 25 = 6.000.000 eventos/día - Eventos mensuales (×30) = 180.000.000 eventos/mes - Definiciones de proceso = 300 × 25 = 7.500 definiciones

Crecimiento relativo (proyectado / inicial) - Tenants: 300 / 50 = 6× - Instancias diarias: 240.000 / 10.000 = 24× - Eventos diarios: 6.000.000 / 150.000 = 40× - Pico de eventos/seg: 2.000 / 300 ≈ 6,7×

Tabla completa

Métrica	Escenario inicial	Escenario proyectado	Observación
Tenants activos	50	300	Crecimiento 6×
Instancias diarias totales	10.000	240.000	Crecimiento 24×
Eventos diarios totales	150.000	6.000.000	Crecimiento 40×: el dato que más crece
Eventos mensuales aproximados	4.500.000	180.000.000	Considerar 30 días
Pico de eventos por segundo	300	2.000	Crecimiento ≈6,7×: presión sobre el motor de escritura

Respuestas a las preguntas

¿Qué datos crecerán más rápido?

Los eventos de auditoría (40×), seguidos de las instancias (24×). Crecen mucho más rápido que los tenants (6×) porque su volumen es multiplicativo: más tenants × más instancias por tenant × más eventos por instancia. Las definiciones de proceso crecen lento (de 500 a 7.500) y se mantienen estables tras su publicación.

¿Qué componente podría convertirse en cuello de botella?

El subsistema de eventos, por dos motivos: (1) volumen acumulado (180 M/mes ≈ 2.190 M/año, crece indefinidamente) y (2) pico de escritura (2.000 eventos/seg). Es un workload write-intensive que un único nodo relacional no absorbe. En segundo lugar, las lecturas de estado de instancia por la concurrencia de usuarios (2.000 concurrentes).

¿Qué datos deberían particionarse desde el inicio?

Eventos e instancias, porque son los de mayor volumen y escritura. Conviene diseñarlos particionados por tenant_id (+ instance_id / created_at) desde el día uno para evitar una migración costosa después.

¿Qué datos podrían permanecer centralizados en una primera versión? - Definiciones de proceso (500-7.500 documentos, bajo volumen, lectura intensiva pero cacheable). - Configuración de tenants (50-300 registros). - Métricas agregadas (se derivan de eventos y pueden recalcularse). Pueden vivir centralizadas/replicadas sin particionar en la etapa inicial.

2. Parte B: Estrategia de particionamiento

Tipo de dato	Clave candidata	Estrategia de particionamiento	Justificación
Tenants	tenant_id	Sin particionar / replicación full (o hash si crece mucho)	Pocos registros (cientos). Conviene replicarlo completo en todos los nodos para lecturas locales de configuración y branding.
Definiciones de procesos	tenant_id + process_id	Hash por tenant_id	Volumen bajo y estable. El hash distribuye uniformemente y mantiene juntas las definiciones de un mismo tenant (consultas siempre scoped por tenant).
Instancias	tenant_id + instance_id	Hash por tenant_id (sub-partición por instance_id)	Alto volumen de escritura. El hash por tenant reparte la carga; las consultas de estado son siempre por instancia dentro de un tenant, así que no necesitan cross-shard.
Tareas humanas	tenant_id + assigned_role	Hash por tenant_id + índice secundario por assigned_role	Se consultan "tareas pendientes de mi rol en mi tenant". Particionar por tenant mantiene localidad; el rol funciona mejor como índice/clustering que como clave de partición (cardinalidad baja → riesgo de hotspot).

Tipo de dato	Clave candidata	Estrategia de particionamiento	Justificación
Eventos	tenant_id + instance_id + timestamp	Hash por (tenant_id,instance_id) + range por timestamp	Append-only de altísimo volumen. La clave compuesta distribuye escrituras y agrupa por instancia; el range temporal permite archivar/consultar por ventanas de tiempo (time-series).
Métricas	tenant_id + date	Range por date (particionado temporal)	Agregados derivados. El range por fecha facilita consultas por período y el archivado de datos viejos sin tocar particiones activas.

Preguntas guía

1. ¿Particionar por tenant_id alcanza?
Como estrategia base, sí: el dominio es multi-tenant y casi toda consulta está scoped por tenant, lo que da localidad y evita cross-shard. Pero no alcanza por sí sola cuando un tenant concentra mucho tráfico (ver Parte C): ahí hay que sub-particionar dentro del tenant.
2. ¿Qué riesgo aparece si un tenant genera mucho más tráfico que los demás?
Hot partition / skew: la partición de ese tenant recibe desproporcionadamente más escrituras y lecturas, saturando ese nodo mientras los demás están ociosos. Rompe el balanceo y degrada la latencia global.
3. ¿Cómo evitarían hot partitions?
 - Clave de partición compuesta (tenant_id + instance_id o + timestamp) para repartir un tenant grande entre varias particiones.
 - Consistent hashing para rebalancear con mínimo movimiento de datos al agregar nodos.
 - Salting / bucketing del tenant dominante (sufijo de bucket en la clave).
 - Sub-partición temporal para los eventos.
4. ¿Qué consultas se benefician de particionar por tenant_id?
Todas las scoped por tenant: listar procesos del tenant, instancias activas del tenant, tareas pendientes por rol, eventos de una instancia. Se resuelven dentro de una sola partición (single-shard), con baja latencia y sin scatter-gather.
5. ¿Qué consultas se complican si los datos quedan demasiado distribuidos?
 - Reportes globales cross-tenant (ej. métricas de toda la plataforma) → requieren cross-shard / scatter-gather.
 - Reconstruir el historial completo de una instancia si sus eventos quedaron repartidos por timestamp en muchas particiones.
 - Cualquier consulta que no incluya la clave de partición → se convierte en un scan de todas las particiones.

3. Parte C: Análisis de hotspots

Caso 1: Tenant dominante (un tenant genera el 40% de los eventos)

1. ¿Qué partición se sobrecarga?
La partición asignada al hash(tenant_id) de ese tenant: concentra el 40% de las escrituras de eventos en un solo nodo.

2. ¿Particionar solo por `tenant_id` sigue siendo suficiente?
No. Con esa clave todos los datos del tenant caen en la misma partición → hotspot garantizado.
3. ¿Qué segunda clave agregarían?
Una clave compuesta que reparta *dentro* del tenant: `tenant_id + instance_id` (o + hash de instancia), de modo que las miles de instancias del tenant grande se distribuyan entre varias particiones.
4. ¿Usarían fecha, hash de instancia, tipo de evento u otra clave?
 - Hash de instancia: mejor para repartir escritura de forma uniforme (recomendado para eventos).
 - Fecha: útil como sub-partición secundaria para archivado/consultas temporales.
 - Tipo de evento: descartado como clave de partición (baja cardinalidad → vuelve a generar hotspots).

Caso 2: Proceso masivo (un proceso de reclamos genera muchas instancias en un día)

1. ¿Conviene particionar por `process_id`?
No como clave primaria: concentraría todas las instancias de ese proceso en una partición (mismo problema de hotspot). Mejor mantener `tenant_id + instance_id` y usar `process_id` como índice secundario.
2. ¿Qué problema aparece si se consulta todo el historial de una instancia?
Si los eventos están particionados por hash/tiempo, el historial completo puede quedar repartido en varias particiones → consulta multi-shard más lenta. Se mitiga agrupando los eventos por `instance_id` (clustering) para que el historial de una instancia quede contiguo.
3. ¿Cómo equilibrar consultas rápidas por instancia y distribución de escritura?
Clave compuesta (`tenant_id, instance_id`) como partición + `timestamp` como clustering key: la escritura se reparte por instancia (alta concurrencia) y la lectura del historial de una instancia es secuencial dentro de su partición (rápida). Es el patrón clásico de Cassandra para event logs.

Caso 3: Eventos históricos (la auditoría crece indefinidamente)

1. ¿Qué estrategia para distribuir eventos por tiempo?
Partición temporal por rangos (ej. por mes: `eventos_2026_06`), combinada con el hash por instancia. Permite “rotar” particiones y mover las viejas a almacenamiento frío.
2. ¿Qué datos deberían quedar en almacenamiento activo (hot)?
Eventos recientes (últimos 30-90 días) y eventos de instancias en curso, que se consultan para operar y debuggear.
3. ¿Qué datos podrían archivar (cold)?
Eventos de instancias ya finalizadas y con más de N meses de antigüedad → mover a almacenamiento de bajo costo (object storage / cold tier), manteniendo solo metadatos indexados.
4. ¿Qué consultas deberían seguir siendo eficientes?
 - (a) Historial de una instancia activa, (b) eventos recientes de un tenant, (c) consultas por ventana temporal reciente. El particionado temporal garantiza que estas consultas toquen pocas particiones calientes.

4. Parte D: Replicación y clustering conceptual

Componente	Tipo de replicación sugerida	Motivo
Definiciones de proceso publicadas	Síncrona	Al publicar una versión, todos los nodos deben verla idéntica antes de aceptar nuevas instancias. Es de baja frecuencia de escritura, así que el costo de la síncrona es asumible y garantiza consistencia fuerte.

Componente	Tipo de replicación sugerida	Motivo
Estado actual de instancia	Síncrona (o semi-síncrona con quórum)	Es la fuente de verdad del workflow. Un estado replicado de forma asíncrona podría leerse stale y producir doble aprobación / transiciones inválidas. Se prioriza consistencia (CP).
Eventos de auditoría	Asíncrona	Append-only y write-intensive. La replicación asíncrona maximiza el throughput de escritura; se tolera un pequeño delay porque el orden causal se preserva con timestamp/vector clocks.
Métricas operativas	Asíncrona	Agregados derivados, no críticos al segundo. La asíncrona prioriza disponibilidad y rendimiento de escritura.
Configuración visual del tenant	Asíncrona	Logo/tema/colores pueden propagarse con segundos de delay sin impacto en la lógica de negocio.

Topología seleccionada

Híbrida: Active-Passive (master-slave) para los datos consistentes + Active-Active para los datos AP.

- Estado de instancia y definiciones de proceso → Active-Passive / master-slave: un nodo primario acepta escrituras y replica a réplicas de lectura. Simplifica la consistencia (un solo punto de escritura, sin conflictos) a costa de disponibilidad ante caída del primario (failover).
- Eventos, métricas y configuración → Active-Active: múltiples nodos aceptan escrituras concurrentes; al ser append-only / derivados, los conflictos son raros o reconciliables. Maximiza disponibilidad y absorbe el pico de escritura distribuido geográficamente.

Justificación (preguntas guía)

1. ¿Qué topología mejora disponibilidad?
Active-active: sin nodo único de escritura, cualquier réplica responde; ideal para eventos y métricas (AP).
2. ¿Cuál simplifica consistencia?
Master-slave / active-passive: un único punto de escritura elimina conflictos de concurrencia; ideal para estado de instancia y definiciones (CP).
3. ¿Cuál es más realista para una primera versión del TPO?
Active-passive (master-slave) para todo: es la más simple de implementar y razonar, suficiente para el volumen inicial (50 tenants), y permite evolucionar hacia híbrida después.
4. ¿Qué topología NO implementarían todavía y por qué?
Active-active multi-región con resolución de conflictos (CRDTs / merge complejo): añade complejidad operativa (detección y resolución de conflictos, latencia inter-región) injustificada para la etapa inicial. Se difiere hasta tener escala real y SLAs que lo exijan.

5. Parte E: Parámetros N, R, W

Recordatorio: con $R + W > N$ se garantiza solapamiento de quóruns de lectura y escritura → consistencia fuerte. Con $R + W \leq N$ → consistencia eventual (mayor disponibilidad/rendimiento).

1. Estado actual de instancia

Parámetro	Valor propuesto	Justificación
N	3	Tres réplicas: balance entre durabilidad y costo.
R	2	Lectura por quórum: garantiza leer el último valor confirmado.
W	2	Escritura por quórum: confirma en mayoría antes de responder.
¿R + W > N?	Sí (2 + 2 = 4 > 3)	Quórums solapados → no hay lecturas stale.
Modelo resultante	Strong	Es el corazón del workflow: se prioriza consistencia sobre disponibilidad.

2. Eventos de auditoría

Parámetro	Valor propuesto	Justificación
N	3	Tres réplicas para durabilidad (no se pueden perder eventos).
R	1	Lectura rápida desde una réplica; el historial tolera leer una versión casi-al-día.
W	1	Escritura rápida en una réplica (luego se propaga async): clave para absorber 2.000 eventos/seg.
¿R + W > N?	No (1 + 1 = 2 ≤ 3)	Se acepta consistencia eventual a cambio de máximo throughput.
Modelo resultante	Eventual	Append-only + orden causal preservado; la durabilidad la da N=3, no el quórum.

3. Métricas operativas

Parámetro	Valor propuesto	Justificación
N	2	Dos réplicas: las métricas son recalculables desde eventos, no requieren alta durabilidad.
R	1	Lectura rápida para dashboards; tolera datos de hace segundos/minutos.
W	1	Escritura asíncrona, sin bloquear el flujo principal.
¿R + W > N?	No (1 + 1 = 2 = 2, no > 2)	Consistencia eventual deliberada.
Modelo resultante	Eventual	Agregados derivados; se prioriza disponibilidad y bajo costo.

Preguntas guía

- ¿Dónde conviene escritura rápida?
En eventos (W=1) y métricas (W=1): write-intensive, toleran eventualidad.
- ¿Dónde conviene lectura consistente?
En estado de instancia (R=2, quórum): antes de decidir una transición hay que ver el valor real.

- ¿Dónde se tolera inconsistencia temporal?
En métricas, configuración de tenant y visualización de historial para usuarios no activos.
- ¿Qué configuración para evitar pérdida de auditoría?
N=3 con W≥1 (idealmente W=2 para durabilidad ante fallo): muchas réplicas garantizan que el evento sobreviva aunque caiga un nodo.
- ¿Qué configuración para dashboards operativos?
N=2, R=1, W=1 → eventual, rápida y barata; los números pueden recalcularse desde el event log.

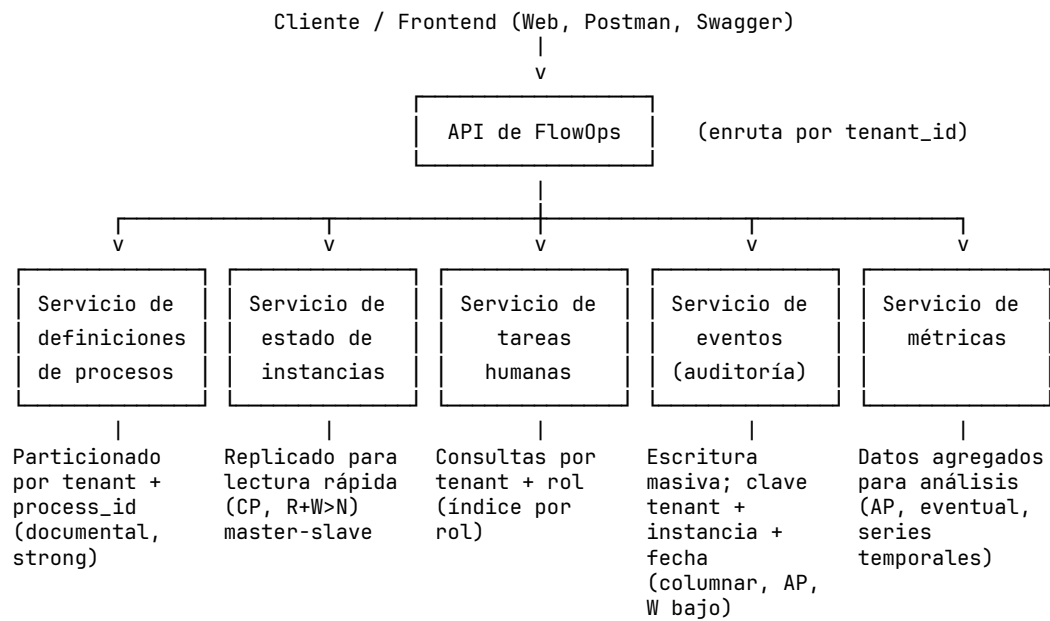
6. Parte F: Plan preliminar de escalabilidad

Etapas	Escenario	Estrategia de escalamiento	Riesgo principal	Acción preventiva
Etapas 1	50 tenants (~10 K instancias/día, ~150 K eventos/día)	Escalamiento vertical + una sola instancia de cada motor NoSQL. Replicación master-slave simple para lectura.	Cuello de botella temprano en escritura de eventos si un tenant crece. Punto único de fallo.	Diseñar las claves de partición desde ya (tenant_id + instance_id, tenant_id + timestamp) aunque todo viva en un nodo, para migrar sin rediseñar. Backups y réplica de lectura.
Etapas 2	300 tenants (~240 K instancias/día, ~6 M eventos/día, pico 2.000 ev/seg)	Escalamiento horizontal: sharding por tenant_id + separación por componentes (motor documental para procesos/instancias, columnar para eventos, KV para estado/cache).	Hot partitions por tenant dominante; el event log empieza a dominar el almacenamiento.	Consistent hashing para rebalanceo barato; sub-partición del tenant grande (salting); archivado temporal de eventos viejos (hot/cold). Réplicas de lectura para los 2.000 usuarios concurrentes.
Etapas 3	+1.000 tenants	Escalamiento horizontal masivo + rebalanceo dinámico + particionado geográfico opcional. Active-active para datos AP.	Rebalanceo costoso; cross-shard queries en reportes globales; crecimiento indefinido del event log.	Rebalanceo automático con consistent hashing; tiering de almacenamiento (hot/warm/cold) y TTL/archivado de eventos; pre-agregación de métricas para evitar scatter-gather; monitoreo de skew por partición.

Consideraciones transversales aplicadas: escalamiento vertical (etapas 1) → horizontal (2 y 3); separación por componentes (cada subsistema en su motor); crecimiento del event log mitigado con tiering y archivado; lecturas frecuentes de estado servidas por réplicas/cache (Redis); escrituras masivas de eventos absorbidas

por motor columnar con W bajo; particiones calientes mitigadas con claves compuestas y consistent hashing; rebalanceo dinámico en etapa 3.

7. Parte G: Arquitectura distribuida conceptual



Mapeo conceptual de modelos NoSQL (alineado con el TPO):

Servicio	Modelo	Partición	Replicación	N/R/W
Definiciones de proceso	Documental	tenant_id + process_id (hash)	Síncrona, master-slave	Strong
Estado de instancia	Documental / Clave-valor	tenant_id + instance_id (hash)	Síncrona/semi (quórum)	N3 R2 W2 (strong)
Tareas humanas	Documental / Clave-valor	tenant_id + índice por rol	Síncrona	Strong (optimistic lock)
Eventos	Columnar / tabular	(tenant_id, instance_id) + timestamp (hash + range)	Asíncrona, active-active	N3 R1 W1 (eventual)
Métricas	Multidimensional / series temporales	tenant_id + date (range)	Asíncrona	N2 R1 W1 (eventual)

8. Puesta en común (exposición oral, 4 min)

Pregunta: ¿Cuál es el dato más difícil de escalar en FlowOps: estado actual, eventos, tareas o métricas?

Respuesta: los EVENTOS de auditoría.

- Por qué: crecen 40× (de 150 K a 6 M/día; ~2.190 M/año) y crecen indefinidamente, con un pico de escritura de 2.000 ev/seg. Es el único dato con crecimiento ilimitado y write-intensive simultáneamente.
- Estrategia de partición: clave compuesta (tenant_id, instance_id) para la partición + timestamp como clustering/range key → reparte la escritura y permite archivar por tiempo (hot/cold tiering).
- Decisión N/R/W: N=3, R=1, W=1 (eventual): máxima velocidad de escritura, durabilidad asegurada por las 3 réplicas; W=2 si se exige no perder ningún evento ante caída de un nodo.

- Consistencia vs disponibilidad: se elige AP / eventual porque los eventos son append-only y el orden causal se preserva con timestamps/vector clocks; no necesitan consistencia fuerte como el estado de instancia.
- Riesgo de hotspot: un tenant dominante (40% de los eventos) satura su partición → se mitiga con sub-partición por instancia (salting) y consistent hashing para rebalanceo.

9. Decisiones que quedan pendientes para las clases de motores específicos

- Elección final de motores concretos (MongoDB para documental, Cassandra/Redis para eventos y estado, InfluxDB para métricas) y su configuración de cluster.
- Esquemas físicos detallados, índices secundarios y CQL/queries reales.
- Implementación de clusters reales, Docker Compose y conectividad desde la aplicación.
- Política exacta de TTL/archivado y benchmarking real de throughput y latencia.
- Tuning fino de N/R/W por motor según sus primitivas de consistencia.